

ENIAD
POO en Java TP3
(Héritage, Polymorphisme et Abstraction)

Exercice 1 :

1. Définir une classe Rectangle avec un constructeur donnant des valeurs (longueur et largeur) par défaut et un attribut nom = "rectangle", une méthode d'affichage et une méthode surface renvoyant la surface d'une instance.
2. Définir une classe Carre héritant de Rectangle et qui surcharge l'attribut d'instance : nom = "carré". Dans le programme principal, instanciez un Rectangle et un Carre et affichez-les.
4. Définir une classe Point avec un constructeur fournissant les coordonnées par défaut d'un point du plan (par exemple : x = 0.0 et y = 0.0). Définir une classe Segment dont le constructeur possède quatre paramètres : deux pour l'origine et deux pour l'extrémité. Ce constructeur définit deux attributs : orig et extrem, instances de la classe Point. De cette manière, vous concevez une classe composite : La classe Segment est composée de deux instances de la classe Point. Ajouter une méthode d'affichage. Enfin écrire un auto-test qui affiche une instance de Segment initialisée par les valeurs 1, 2, 3 et 4.

Exercice 2 :

Ecrivez une classe Bâtiment avec les attributs suivants: adresse

La classe Bâtiment doit disposer des constructeurs suivants:

Batiment(),

Batiment (adresse).

La classe Bâtiment doit contenir des accesseurs et mutateurs (ou propriétés) pour les différents attributs. La classe Bâtiment doit contenir une méthode *afficher ()* donnant une représentation du Bâtiment.

Ecrivez une classe Maison héritant de Bâtiment avec les attributs suivants: NbPieces: Le nombre de pièces de la maison.

La classe Maison doit disposer des constructeurs suivants:

Maison(),

Maison(adresse, nbPieces).

La classe Maison doit contenir des accesseurs et mutateurs (ou des propriétés) pour les différents attributs. La classe Maison doit contenir une méthode *afficher ()* donnant une représentation de la Maison.

Ecrivez aussi un programme afin de tester ces deux classes.

Exercice 3 :

Soit la classe abstraite Employé caractérisée par attributs suivants :

- Matricule
- Nom
- Prénom
- Date de naissance

La classe Employé doit disposer des méthodes suivantes :

- un *constructeur* d'initialisation
- des propriétés pour les différents attributs
- la méthode *afficher*
- une méthode abstraite *GetSalaire*.

Un ouvrier est **un employé** qui se caractérise par sa date d'entrée à la société.

- Tous les ouvriers ont une valeur commune appelée SMIG=2500 DH
- L'ouvrier a un salaire mensuel qui est : $\text{Salaire} = \text{SMIG} + (\text{Ancienneté en année}) \times 100$.
- De plus, le salaire ne doit pas dépasser $\text{SMIG} \times 2$.

Un cadre est **un employé** qui se caractérise par un indice.

- Le cadre a un salaire qui dépend de son indice :
 - 1 : salaire mensuel 13000 DH
 - 2 : salaire mensuel 15000 DH
 - 3 : salaire mensuel 17000 DH
 - 4 : salaire mensuel 20000 DH

Un patron est **un employé** qui se caractérise par un chiffre d'affaire et un pourcentage.

- Le chiffre d'affaire est commun entre les patrons.
- Le patron a un salaire annuel qui est égal à x% du chiffre d'affaire : $\text{Salaire} = \text{CA} \times \text{pourcentage} / 100$

Travail à faire:

1. Créer la classe abstraite Employé.
2. Créer la classe Ouvrier, la classe Cadre et la classe Patron qui héritent de la classe Employé, et prévoir les Constructeurs et la méthode *afficher* de chacune des 3 classes.
3. Implémenter la méthode *GetSalaire()* qui permet de calculer le salaire pour chacune des classes.

Note : La syntaxe pour créer une classe abstraite `AbstractClassExample` en java, avec la méthode abstraite `methode1()`:

```
public abstract class AbstractClassExample { // déclarée dans le fichier A.java
    public abstract void methode1() ; // pas de corps de méthode
}
```

Exercice 4 :

Un parc auto se compose des voitures et des camions qui ont des caractéristiques communes regroupées dans la classe Véhicule.

- Chaque véhicule est caractérisé par son matricule, l'année de son modèle, son prix.
- Lors de la création d'un véhicule, son matricule est incrémenté selon le nombre de véhicules créés.
- Tous les attributs de la classe véhicule sont supposés privés. ce qui oblige la création des accesseurs (`get...`) et des mutateurs (`set...`) ou les propriétés.
- La classe Véhicule possède également deux méthodes abstraites `démarrer()` et `accélérer()` qui seront définies dans les classes dérivées et qui afficheront des messages personnalisés.
- La méthode `ToString()` de la classe Véhicule retourne une chaîne de caractères qui contient les valeurs du matricule, de l'année du modèle et du prix.
- Les classes Voiture et Camion étendent la classe Véhicule en définissant concrètement les méthodes `accélérer()` et `démarrer()` en affichant des messages personnalisés.

1. Créer la classe abstraite Véhicule
2. Créer les classes Camion et Voiture
3. Créer un tableau de Véhicules, contenant une Voiture et un Camion, puis afficher les caractéristiques de chaque Véhicule